

UNIVERSIDAD DEL VALLE DE GUATEMALA
CC3069 - COMPUTACIÓN PARALELA Y DISTRIBUIDA
SECCIÓN – 10
Ing. Kimberly Barrera



Práctica Guiada 01

Pedro Rubén Avila Cofino 23089
Javier Alejandro Lopez del Cid 23415
Javier Andre Benitez Garcia 23405

GUATEMALA, 22 de septiembre de 2026

Índice

1. Introducción	2
2. Antecedentes	3
2.1 Programación paralela con memoria compartida y OpenMP	3
2.2 Método PCAM.....	3
2.3 Speedup, eficiencia y ley de Amdahl.....	4
2.4 OpenGL, GLUT y el video Bad Apple.....	4
3. Diseño de la solución	5
3.1 Descripción general	5
3.2 Parámetros de ejecución	5
3.3 Modelo físico	5
3.4 Organización del código	6
4. Paralelización con el método PCAM.....	7
4.1 Análisis del programa secuencial.....	7
4.2 Partición	7
4.3 Comunicación	7
4.4 Aglomeración.....	7
4.5 Mapeo	7
5. Versiones paralelas	8
5.1 Paralelo v1: parallel for y atomic.....	8
5.2 Paralelo v2: mejora iterativa	8
5.3 Mecanismos de sincronía y manejo de memoria	8
6. Resultados	10
6.1 Metodología de medición	10
6.2 Speedup y eficiencia	10
6.3 Discusión.....	10
7. Conclusiones	11
8. Recomendaciones	12
9. Referencias.....	13
Anexo 1. Diagrama de flujo del programa.....	14
Anexo 2. Catálogo de funciones	16
Anexo 3. Bitácora de pruebas	19

1. Introducción

Un screensaver es un programa gráfico que se ejecuta de forma continua y cuyo atractivo depende de que la animación sea fluida. Cuando la cantidad de elementos en pantalla crece, el tiempo necesario para calcular cada cuadro también crece y, si supera los 16.7 ms que permiten 60 cuadros por segundo (FPS), la animación se ve entrecortada. Este proyecto aprovecha esa situación como caso de estudio de programación paralela con memoria compartida.

The Bad Apple Screensaver dibuja N partículas de colores pseudoaleatorios que se mueven, chocan entre sí y rebotan en los bordes de la ventana. Cuando se le proporciona un archivo con la silueta del video *Bad Apple*, las partículas son atraídas hacia la figura de cada cuadro y la reproducen en pantalla. El programa está escrito en C, dibuja con OpenGL y GLUT, y calcula la física en tres versiones: una secuencial y dos paralelas con OpenMP, la segunda obtenida como mejora iterativa de la primera.

El objetivo del informe es documentar el diseño del programa, la aplicación del método PCAM para paralelizarlo, los mecanismos de sincronía empleados y la medición del desempeño mediante speedup y eficiencia. Los anexos contienen el diagrama de flujo, el catálogo de funciones y la bitácora de pruebas.

2. Antecedentes

2.1 Programación paralela con memoria compartida y OpenMP

En un sistema de memoria compartida todos los hilos de un proceso pueden leer y escribir las mismas variables. Esto simplifica la comunicación entre tareas, pero obliga a coordinar los accesos: si dos hilos escriben la misma posición de memoria sin protección se produce una condición de carrera y el resultado depende del orden de ejecución.¹

OpenMP es una interfaz estándar de directivas de compilador, funciones de biblioteca y variables de entorno para C, C++ y Fortran que permite paralelizar un programa secuencial de forma incremental.² Sigue el modelo *fork-join*: el hilo maestro crea un equipo de hilos al entrar a una región `#pragma omp parallel` y los sincroniza al salir. Las construcciones utilizadas en este proyecto son:

- `parallel for`: reparte las iteraciones de un ciclo entre los hilos; al final existe una barrera implícita.
- `schedule(static)`: divide las iteraciones en bloques contiguos de igual tamaño, adecuado cuando todas cuestan lo mismo.
- `atomic`: garantiza que una actualización de memoria (por ejemplo `suma += x`) se realice de forma indivisible.
- `reduction(+:var)`: cada hilo trabaja sobre una copia privada de la variable y OpenMP combina las copias al final, sin contención.
- `single`: un solo hilo ejecuta el bloque mientras los demás esperan en la barrera implícita del final.

2.2 Método PCAM

Foster propone diseñar algoritmos paralelos en cuatro etapas: **Partición** (descomponer el problema en tareas lo más pequeñas posible), **Comunicación** (identificar los datos que deben intercambiar las tareas), **Agglomeración** (agrupar tareas para reducir el costo de comunicación y de creación de hilos) y **Mapeo** (asignar los grupos resultantes a los procesadores).³ El método separa las decisiones que dependen del problema de las que dependen de la máquina.

¹Pacheco y Malensek (2022), cap. 5, describen las condiciones de carrera y las secciones críticas en OpenMP.

²OpenMP Architecture Review Board (2021). Especificación OpenMP 5.2.

³Foster (1995), cap. 2, "Designing Parallel Algorithms".

2.3 Speedup, eficiencia y ley de Amdahl

El speedup mide cuántas veces más rápida es la versión paralela respecto a la secuencial, y la eficiencia indica qué fracción de los p hilos se aprovecha realmente:⁴

$$S(p) = T_{\text{secuencial}} / T_{\text{paralelo}}(p) \quad E(p) = S(p) / p$$

La ley de Amdahl establece que si una fracción f del programa es inherentemente secuencial, el speedup máximo con p procesadores es $S(p) \leq 1 / (f + (1 - f)/p)$.⁵ Por ello, además de paralelizar la parte costosa, conviene reducir lo que queda en serie, como la creación repetida de hilos o la actualización de datos compartidos.

2.4 OpenGL, GLUT y el video Bad Apple

OpenGL es una API de bajo nivel para gráficos 2D y 3D que delega el dibujo a la GPU, mientras que GLUT (freelut en Linux y Windows, GLUT.framework en macOS) crea la ventana y ejecuta el ciclo de eventos.⁶ Esta separación permite que la CPU se dedique al cálculo de la física, que es la parte que se paraleliza. *Bad Apple* es un video en blanco y negro muy usado en demostraciones gráficas porque solo importa su silueta; en este proyecto cada cuadro se reduce a una grilla de bits que sirve como destino de las partículas.

⁴Definiciones según Pacheco y Malensek (2022), sección 2.6.

⁵Amdahl (1967).

⁶Segal y Akeley (2006); The fre glut Project (2022).

3. Diseño de la solución

3.1 Descripción general

El programa recibe como parámetro obligatorio N, la cantidad de partículas, y opcionalmente la versión de la física, el número de hilos, el tamaño del canvas, el radio de las partículas, la semilla pseudoaleatoria y la ruta de la silueta. Todos los valores se validan antes de abrir la ventana. Las partículas se inicializan con posición, dirección y color pseudoaleatorios a partir de la semilla, de modo que las tres versiones simulan exactamente la misma escena y sus tiempos son comparables.

Requisito del enunciado	Implementación
Al menos un parámetro N	Primer argumento posicional (1 a 200 000), más 8 opciones con --
Varios colores pseudoaleatorios	Color HSV aleatorio por partícula (generador xorshift con semilla)
Canvas mínimo de 640x480	--ancho y --alto validados; por defecto 800x600
Movimiento estéticamente interesante	Partículas que forman la silueta de Bad Apple o giran en espiral
Física o trigonometría	Colisiones entre partículas, resorte, amortiguamiento, rebote, atan2/sin/cos
Desplegar los FPS	En pantalla, en el título de la ventana y en consola cada segundo

Tabla 1. Cumplimiento de las reglas básicas del screensaver

3.2 Parámetros de ejecución

```
./bin/screensaver N [--modo seq|par1|par2] [--hilos T] [--ancho W] [--alto H]
                  [--radio R] [--semilla S] [--bap RUTA] [--benchmark F]

./bin/screensaver 4000                                # paralelo v2, todos los núcleos
./bin/screensaver 4000 --modo seq                      # versión secuencial
./bin/screensaver 4000 --modo par1 --hilos 4          # paralelo v1 con 4 hilos
```

La programación defensiva se implementa en `config.c`: los números se convierten con `strtoul/strtouf` verificando que la cadena completa sea numérica y que el valor esté en rango; las opciones desconocidas, los valores faltantes y un canvas menor a 640x480 producen un mensaje de error y la forma de uso. También se verifica cada `malloc` y la integridad del archivo `.bap` (encabezado, dimensiones y tamaño). Si la silueta no existe, el programa continúa en modo de rebotes en lugar de fallar.

3.3 Modelo físico

En cada cuadro se avanza la simulación con un paso fijo $\Delta t = 1/60$ s. Para cada partícula *i* se calcula:

1. **Repulsión (colisiones).** Para toda partícula j a una distancia d menor que el diámetro $D = 2r$ se aplica una fuerza $F = k_{rep} \cdot (D - d) / d \cdot (x_i - x_j)$. Este cálculo recorre las N partículas, por lo que el costo por cuadro es $O(N^2)$ y domina el tiempo de ejecución.
2. **Atracción a la silueta.** Si hay silueta, la partícula i se asigna a la celda encendida número $[i \cdot M / N]$ (M = celdas encendidas del cuadro actual) y se aplica un resorte $F = k_{atr} \cdot (\text{destino} - \text{posición})$. Sin silueta se aplica una fuerza tangencial calculada con atan2 , $\sin$ y $\cos$, más una onda senoidal vertical.
3. **Integración.** Euler semi-implícito: $v \leftarrow (v + F \cdot \Delta t) \cdot \text{amortiguamiento}$ y $x \leftarrow x + v \cdot \Delta t$, limitando la rapidez máxima.
4. **Rebote.** Si la partícula sale del canvas se corrige su posición y se invierte la componente de la velocidad con un coeficiente de restitución de 0.9.

El brillo del color depende de la rapidez, de modo que las partículas rápidas se ven más intensas. El video de 6 572 cuadros se preprocesa con `scripts/extract_frames.py` (Python y OpenCV) a una grilla de 96x72 bits por cuadro, y el programa en C lo dibuja como fondo tenue detrás de las partículas.

3.4 Organización del código

Archivo	Responsabilidad
<code>src/main.c</code>	Ventana GLUT, ciclo principal, dibujo, cálculo de FPS y modo benchmark
<code>src/config.c</code>	Lectura y validación de argumentos
<code>src/sistema.c</code>	Memoria de las partículas, inicialización pseudoaleatoria y silueta
<code>include/fisica.h</code>	Cálculo de una partícula (compartido por las tres versiones)
<code>src/fisica_secuencial.c</code>	Versión secuencial
<code>src/fisica_paralela.c</code>	Versiones paralelas v1 y v2
<code>src/frames.c</code>	Lectura del formato binario .bap
<code>scripts/benchmark.sh</code>	Bitácora automática: 10 corridas por configuración

Tabla 2. Módulos del programa

Las partículas se guardan como estructura de arreglos (un arreglo por propiedad) para que cada hilo recorra memoria contigua. Las tres versiones llaman a las mismas funciones `static inline` de `fisica.h`, de modo que solo cambia la forma en que se reparte el trabajo y cualquier diferencia de tiempo se debe a la paralelización.

4. Paralelización con el método PCAM

4.1 Análisis del programa secuencial

En la versión secuencial cada cuadro ejecuta: (a) la actualización de la lista de celdas de la silueta, $O(96 \cdot 72)$ y solo cuando cambia el cuadro del video; (b) el ciclo de fuerzas, $O(N^2)$; (c) el llenado del buffer de dibujo, $O(N)$, y (d) el dibujo en la GPU. Para $N = 4\,000$ el ciclo de fuerzas evalúa 16 millones de pares por cuadro, por lo que concentra casi todo el tiempo de CPU y es el candidato a paralelizar.

4.2 Partición

Se usó descomposición de dominio sobre los datos: una tarea por partícula, que calcula la fuerza total sobre ella y su nuevo estado. Esto produce N tareas de igual costo (cada una recorre las N partículas). Una descomposición por pares (i, j) daría N^2 tareas más finas, pero exigiría sumar fuerzas de varias tareas sobre la misma partícula, con mucha más sincronía.

4.3 Comunicación

Cada tarea necesita leer la posición de todas las partículas, lo que en memoria compartida es una lectura sin costo de mensajes. El conflicto aparecería si una tarea actualizara la posición de i mientras otra la está leyendo. Para eliminarlo se usa **doble buffer**: todas las tareas leen el estado actual (x, y, vx, vy) y cada una escribe solo su posición i del estado siguiente $(x_sig, y_sig, \dots)$. Al terminar el cuadro se intercambian los punteros. La única comunicación de escritura son dos sumas globales: el número de choques y la energía cinética.

4.4 Aglomeración

Crear un hilo por partícula sería costoso, así que las N tareas se agrupan en bloques contiguos de N/p partículas con `schedule(static)`. Como todas las tareas tienen el mismo costo, el reparto estático equilibra la carga sin el costo de un planificador dinámico y conserva la localidad de memoria. En la versión 2 se aglomeran además el cálculo de la física y el llenado del buffer de dibujo en una sola iteración.

4.5 Mapeo

Se asigna un bloque por hilo y el número de hilos se controla con `--hilos` (por defecto, la cantidad de núcleos). El sistema operativo y el runtime de OpenMP asignan los hilos a los núcleos físicos.

5. Versiones paralelas

5.1 Paralelo v1: parallel for y atomic

La primera versión es la transformación directa del código secuencial: un `parallel for` sobre el ciclo de fuerzas y otro sobre el buffer de dibujo. Las sumas globales se protegen con `atomic`, que es correcto pero hace que todos los hilos compitan por la misma variable en cada iteración.

```
#pragma omp parallel for num_threads(hilos) schedule(static)
for (int i = 0; i < s->n; i++) {
    float fx = 0.0f, fy = 0.0f;
    int c = fuerza_repulsion(s, i, &fx, &fy); /* O(N) por particula */
    float e = integrar_particula(s, i, fx, fy, t); /* escribe solo en i */
#pragma omp atomic
    colisiones += c;
#pragma omp atomic
    energia += e;
} /* barrera implicita */
#pragma omp parallel for num_threads(hilos) schedule(static)
for (int i = 0; i < s->n; i++) llenar_buffer_render(s, i);
```

5.2 Paralelo v2: mejora iterativa

La segunda versión aplica cuatro cambios sobre la v1: (1) una sola región paralela por cuadro, para crear el equipo de hilos una vez y no dos; (2) la actualización de la silueta dentro de la región con `single`, cuya barrera implícita garantiza que nadie lea la lista de celdas mientras se escribe; (3) `reduction(+:colisiones, energia)` en lugar de `atomic`, eliminando la contención, y (4) la fusión del ciclo de dibujo con el de física, que reduce una barrera y aprovecha que los datos de la partícula `i` ya están en caché.

```
#pragma omp parallel num_threads(hilos) reduction(+ : colisiones, energia)
{
#pragma omp single
    sistema_actualizar_silueta(s, t); /* un hilo; barrera al final */
#pragma omp for schedule(static)
    for (int i = 0; i < s->n; i++) {
        float fx = 0.0f, fy = 0.0f;
        colisiones += fuerza_repulsion(s, i, &fx, &fy);
        energia += integrar_particula(s, i, fx, fy, t);
        llenar_buffer_render(s, i); /* ciclo fusionado */
    }
} /* barrera implicita y combinacion de la reduccion */
```

5.3 Mecanismos de sincronía y manejo de memoria

Mecanismo	Dónde	Qué protege
Doble buffer	v1 y v2	Evita condiciones de carrera entre lecturas del estado actual y escrituras del siguiente
atomic	v1	Sumas globales de choques y energía
reduction(+)	v2	Las mismas sumas, con copias privadas por hilo

Mecanismo	Dónde	Qué protege
single + barrera implícita	v2	Lista de celdas de la silueta, escrita por un hilo antes de que los demás la lean
Barrera implícita de for	v1 y v2	Que todas las partículas tengan su nuevo estado antes de intercambiar buffers y dibujar

Tabla 3. Mecanismos de sincronía utilizados

Toda la memoria se reserva una sola vez en `sistema_crear` y se libera en `sistema_destruir`, registrada con `atexit` porque `glutMainLoop` no retorna en todas las implementaciones de GLUT. Si cualquier reserva falla se libera lo ya reservado y el programa termina con un mensaje de error.

6. Resultados

6.1 Metodología de medición

El modo `--benchmark F` ejecuta $F = 120$ cuadros con la ventana abierta y sin límite de FPS, mide con `omp_get_wtime` el tiempo de la física de cada cuadro y el tiempo total, e imprime una línea CSV. El script `scripts/benchmark.sh` repite cada configuración 10 veces para $N = 1\,000, 2\,000, 4\,000$ y $8\,000$, con la versión secuencial y las dos paralelas con distintos números de hilos. El speedup se calcula con el **promedio** de las 10 corridas del tiempo de física por cuadro, y la eficiencia como speedup entre hilos.

6.2 Speedup y eficiencia

[PENDIENTE] Tabla y gráficas de speedup/eficiencia: correr `./scripts/benchmark.sh`

6.3 Discusión

[PENDIENTE] Discusión de resultados

7. Conclusiones

- [PENDIENTE] Conclusiones basadas en las mediciones.

8. Recomendaciones

- Reemplazar la búsqueda de vecinos $O(N^2)$ por una grilla espacial (celdas del tamaño del diámetro), que reduce el costo a aproximadamente $O(N)$ y permitiría más partículas a 60 FPS; la grilla también se puede construir en paralelo.
- Usar vertex buffer objects o shaders para el dibujo, porque cuando la física ya es rápida el tiempo del cuadro queda dominado por la GPU y la sincronización vertical.
- Medir con más repeticiones y descartar las primeras corridas de calentamiento para reducir la variación causada por otros procesos del sistema.
- Probar `schedule(dynamic)` o `guided` si se agregan fuerzas cuyo costo varíe entre partículas, ya que el reparto estático solo es óptimo con carga uniforme.

9. Referencias

- Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. En AFIPS Spring Joint Computer Conference Proceedings (Vol. 30, pp. 483–485). ACM. <https://doi.org/10.1145/1465482.1465560>
- Chandra, R., Dagum, L., Kohr, D., Maydan, D., McDonald, J., & Menon, R. (2001). Parallel programming in OpenMP. Morgan Kaufmann.
- Foster, I. (1995). Designing and building parallel programs: Concepts and tools for parallel software engineering. Addison-Wesley.
- OpenMP Architecture Review Board. (2021). OpenMP application programming interface (Version 5.2). <https://www.openmp.org/specifications/>
- Pacheco, P. S., & Malensek, M. (2022). An introduction to parallel programming (2nd ed.). Morgan Kaufmann.
- Segal, M., & Akeley, K. (2006). The OpenGL graphics system: A specification (Version 2.1). Silicon Graphics / Khronos Group.
<https://registry.khronos.org/OpenGL/specs/gl/glspec21.pdf>
- The freeglut Project. (2022). freeglut: The free OpenGL utility toolkit (Version 3.4). <https://freeglut.sourceforge.net/>

Anexo 1. Diagrama de flujo del programa

El diagrama muestra la captura de argumentos, la solicitud de datos (archivo de silueta), los puntos de programación defensiva, las secciones paralelas, los mecanismos de sincronía y el despliegue de resultados.

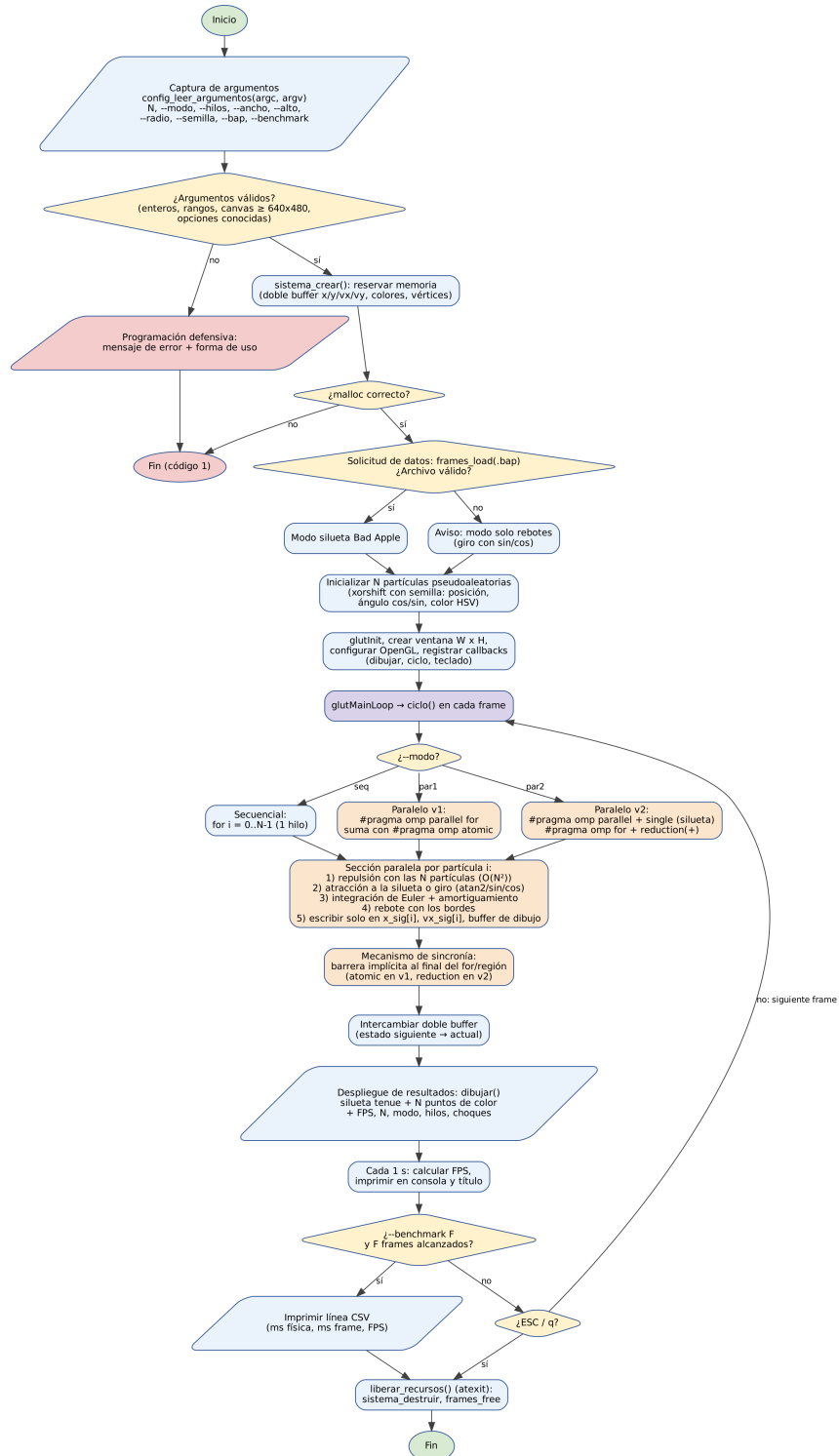


Figura A1. Diagrama de flujo de The Bad Apple Screensaver

Anexo 2. Catálogo de funciones

Para cada rutina se indican sus entradas (nombre y tipo), sus salidas y su propósito.

Función (archivo)	Entradas	Salidas	Descripción
main (main.c)	argc: int, argv: char**	int: 0 si termina bien, 1 si hay error	Valida argumentos, crea el sistema, abre la ventana GLUT, registra callbacks e inicia el ciclo principal.
ciclo (main.c)	— (usa g_cfg, g_sistema)	—	Callback idle: ejecuta un paso de fisica, pide redibujar, calcula los FPS cada segundo y controla el modo benchmark.
ejecutar_paso_fisica (main.c)	— (usa g_cfg.modos)	double: segundos del paso	Llama a la versión secuencial, v1 o v2 y mide su duración con omp_get_wtime.
dibujar (main.c)	—	—	Dibuja la silueta tenue, las N partículas con vertex arrays (una sola llamada) y el panel de FPS.
dibujar_texto (main.c)	x, y: float posición; texto: const char*	—	Escribe texto en pantalla con glutBitmapCharacter.
terminar_benchmark (main.c)	—	Línea CSV en stdout	Calcula ms de fisica, ms por cuadro y FPS promedio, los imprime y termina el programa.
teclado (main.c)	tecla: unsigned char; x, y: int	—	Cierra el programa con ESC o q.
redimensionar (main.c)	w, h: int tamaño de ventana	—	Ajusta el viewport manteniendo coordenadas en pixeles del canvas.
configurar_opengl (main.c)	—	—	Color de fondo, puntos suavizados, tamaño del punto y arreglos de vértices/colores.
tiempo_actual (main.c)	—	double: segundos	Reloj de pared (omp_get_wtime o clock_gettime).
liberar_recursos (main.c)	—	—	Registrada con atexit; libera el sistema al salir.
config_leer_argumentos (config.c)	argc: int; argv: char**; cfg: Config*	int: 0 válido, 1 ayuda, -1 error; llena cfg	Asigna valores por defecto, interpreta N y las opciones y valida rangos.
leer_entero/leer_flotante (config.c)	nombre, texto: const char*; minimo, maximo	int: 0/-1; valor en salida	Convierten texto a número con strtol/strtod verificando formato y rango.
config_imprimir_uso (config.c)	programa: const char*	Texto en stderr	Muestra la forma de uso y las opciones.
config_nombre_modos (config.c)	modo: ModoEjecucion	const char*	Nombre legible del modo para pantalla y CSV.
sistema_crear (sistema.c)	cfg: const Config*	Sistema* o NULL si falla	Reserva memoria, inicializa posiciones, velocidades (cos/sin) y colores HSV pseudoaleatorios y carga la silueta.

Función (archivo)	Entradas	Salidas	Descripción
sistema_destruir(sistema.c)	s: Sistema* (acepta NULL)	—	Libera todos los arreglos y la silueta.
sistema_actualizar_silueta(sistema.c)	s: Sistema*; t: double tiempo simulado	Actualiza pix_x, pix_y, num_pix, fondo	Obtiene el cuadro del video para el tiempo t y genera la lista de celdas encendidas en coordenadas de pantalla.
sistema_intercambiar_buffers(sistema.c)	s: Sistema*	—	Intercambia los punteros del estado actual y el siguiente (doble buffer).
rng_sembrar/rng_uniforme(sistema.c)	semilla: unsigned int	float en [0, 1)	Generador pseudoaleatorio xorshift32 reproducible.
hsv_a_rgb(sistema.c)	h, s, v: float	r, g, b: float*	Convierte un color HSV a RGB.
fuerza_repulsion(fisica.h)	s: const Sistema*; i: int	int choques con j > i; suma en fx, fy	Recorre las N partículas y acumula la fuerza de repulsión de las que tocan a i (O(N)).
integrar_particula(fisica.h)	s: Sistema*; i: int; fx, fy: float; t: double	float: energía cinética de i	Suma atracción o giro, integra con Euler, aplica amortiguamiento, límite de rapidez y rebote; escribe solo la posición i del estado siguiente.
llenar_buffer_render(fisica.h)	s: Sistema*; i: int	Escribe vertices[i], colores[i]	Copia la posición y calcula el color con brillo según la rapidez.
fisica_paso_secuencial(fisica_secuencial.c)	s: Sistema*; t: double	Actualiza s	Un paso completo en un solo hilo (línea base).
fisica_paso_paralelo_v1(fisica_paralela.c)	s: Sistema*; t: double; hilos: int	Actualiza s	Dos parallel for y sumas con atomic.
fisica_paso_paralelo_v2(fisica_paralela.c)	s: Sistema*; t: double; hilos: int	Actualiza s	Una región paralela con single, for y reduction; ciclos fusionados.
frames_load(frames.c)	path: const char*	Frames* o NULL	Lee y valida un archivo .bap (encabezado BADA, dimensiones, tamaño).
frames_free(frames.c)	frames: Frames*	—	Libera la silueta.
frames_get_pixel(frames.c)	frames; frame_idx, x, y: uint32_t	int: 1 si la celda es silueta	Lee un bit de la grilla empacada.

Tabla A2.1. Funciones del programa

Estructura	Campos	Uso
Config	num_particulas, modo, num_hilos, ancho, alto, radio, semilla, ruta_bap, frames_benchmark	Configuración leída de la línea de comandos.
Sistema	n; x, y, vx, vy (actual); x_sig, y_sig, vx_sig, vy_sig (siguiente); color_base, vertices, colores; frames, pix_x, pix_y, num_pix, fondo; colisiones, energia	Estado completo de la simulación como estructura de arreglos.
Frames	cols, rows, frame_count, fps, data	Silueta del video cargada en memoria.

Tabla A2.2. Estructuras de datos

Anexo 3. Bitácora de pruebas

[PENDIENTE] Tablas con las 10 mediciones por prueba (se generan desde results/benchmark.csv)

Capturas de las mediciones

[PENDIENTE] Captura de la terminal con el benchmark y del screensaver corriendo